

High-Recall BEC/Phishing Detection Pipeline

This project implements a complete Machine Learning pipeline for classifying incoming emails as either **Legitimate (0)** or **Malicious (1)**, with a dedicated focus on detecting sophisticated **Business Email Compromise (BEC)** and phishing attempts.

The core design principle of this pipeline is **Security First**. The Random Forest model is aggressively tuned to maximize **Recall** for the malicious class, ensuring that the system prioritizes catching threats even at the expense of a slightly higher False Positive rate.

Key Security Feature: Tuned for Recall

In a security context, a **False Negative (missing an attack)** is far more dangerous than a **False Positive (flagging a safe email)**.

To achieve this high level of threat detection, the training process uses explicit **class weighting** (`{0: 1, 1: 5}`). This tells the model that misclassifying a malicious email is five times worse than misclassifying a legitimate one, resulting in a **Malicious Recall score of 92%**.

Metric	Legitimate (0)	Malicious (1)
Recall	~0.33	~ 0.92
Precision	~0.55	~0.83

Project Architecture

The pipeline follows a standard ML workflow using four Python scripts:

File	Role	Description
<code>data_simulator.py</code>	Data Generation	Creates the synthetic, feature-rich dataset (<code>simulated_emails.csv</code>). Features include <code>urgency_score</code> , <code>domain_similarity_score</code> , and <code>financial_keyword_count</code> .
<code>feature_engineer.py</code>	Preprocessing	Performs data cleaning, One-Hot Encoding on categorical features, and standard train/test splitting.
<code>train_model.py</code>	Training & Tuning	Trains the Random Forest Classifier and applies the critical class weight tuning to maximize threat detection. Saves the final optimized model as <code>model.pkl</code> .
<code>predict_email.py</code>	Deployment Demo	Loads the saved <code>model.pkl</code> and demonstrates classification on a new, unseen email record in a production-like setting.

Getting Started

To run this detection pipeline locally, follow these steps:

1. Setup

Clone the repository and install the necessary Python dependencies:

```
# Clone the repository
git clone [https://github.com/sayuj5/bec-phishing-detection-pipeline.git](https://github.com/sayuj5/bec-phishing-detection-pipeline.git)
cd bec-phishing-detection-pipeline

# Install required libraries
pip install -r requirements.txt
```

2. Execution Pipeline

Run the scripts sequentially to generate data, train the tuned model, and perform a live prediction test:

```
# 1. Generate the synthetic data
python data_simulator.py

# 2. Train the model (applies the class weighting)
python train_model.py

# 3. Test the prediction on a suspicious email
python predict_email.py
```

The final output will confirm that the model correctly flags the suspicious email as **MALICIOUS**.